

Common Vulnerabilities & Public Exploit Resources

*How vulnerabilities are born, and where a pen tester finds
working exploit code*

Prepared for: Goraya — Red Team Intern, CyberStr

Trainer: Umar Niaz

Source: ITProTV - PenTest+ Series (Daniel Lowrie & Ronnie)

Level: Junior Penetration Tester

Topics: Root causes of vulnerabilities, Exploit-DB, SearchSploit, Vulners,
PacketStorm, Google Dorking, GitHub, Exploit Chaining

Notes compiled and expanded for practical, exam-and-lab-ready learning. Explanatory + reasoning style, plain English.

00 Table of Contents

1. Introduction — What This Session Covers	Sec 1
2. Why Vulnerabilities Exist in the First Place	Sec 2
2.1 Lack of Error Handling	Sec 2.1
2.2 Lack of Code Signing	Sec 2.2
2.3 Insecure Data Transmission	Sec 2.3
2.4 Trusting User Input	Sec 2.4
3. Where To Find Public Exploit Code	Sec 3
3.1 Exploit-DB	Sec 3.1
3.2 SearchSploit (Offline Exploit-DB)	Sec 3.2
3.3 Vulners	Sec 3.3
3.4 PacketStorm Security	Sec 3.4
3.5 Google Search / Google Dorking	Sec 3.5
3.6 GitHub	Sec 3.6
4. Exploit Chaining — The Real-World Attack Path	Sec 4
5. Quick Reference & Cheat Sheet	Sec 5
6. Key Takeaways for a Junior Pen Tester	Sec 6

01 Introduction

This lesson answers two practical questions every beginner pen tester asks sooner or later.

1 Why do vulnerabilities exist at all? — understanding the root causes of bugs helps you predict where to look for them.

2 Once I know a vulnerability exists, how do I actually get the exploit code for it? — a list of real, trusted resources used by professional pen testers to find working proof-of-concept (PoC) exploits.

WHY THIS MATTERS

Knowing a vulnerability's name is not the same as being able to exploit it. Many beginners get stuck right here — they can scan and find a CVE, but they don't know where to go next. This session builds that missing bridge: root cause → public resource → working exploit → chained access.

02 Why Vulnerabilities Exist

According to the lesson, almost every common application vulnerability can be traced back to one of four root problems. These are generalizations — real bugs are often messier — but as a mental model, this is extremely useful for a junior pen tester.

Lack of Error Handling

Lack of Code Signing

Insecure Data Transmission

Trusting User Input

2.1 Lack of Error Handling

Error handling means: what does the application do when it receives data it did NOT expect? A well-built app should catch bad input gracefully. A poorly-built app either crashes or does something undefined — and that undefined behaviour is exactly where an attacker looks for an opening.

WALKTHROUGH EXAMPLE (FROM THE LESSON)

Imagine a simple "phonebook" app that asks for: **Name**, **Date of Birth**, and **City**. A normal user types normal values and everything works. But if you type something the app doesn't expect — for example a huge string of digits instead of a name — the app should flag this as invalid input. If it wasn't coded to expect anything odd, the app may crash entirely.

A second, more subtle example: a user's real name is **O'Reilly**. The apostrophe ' is a special character. If the developer only expected plain letters, that single apostrophe can break the input parser — this is actually the same root idea behind classic **SQL Injection**, where a stray quote character breaks out of a database query.

REASONING — WHY THIS CREATES A VULNERABILITY

Every program is built expecting a certain **data type** (letters only, numbers only, a fixed length, etc). When the real-world input does not match that expectation and there is no check for it, the program is forced into a state its developer never designed for. Two outcomes are possible, and both matter to a pen tester:

- **Fail Closed** — the system shuts down / blocks the action. Safer, but can still leak clues (e.g. a stack trace) in the error message.
- **Fail Open** — the system lets the action through anyway, often skipping validation entirely. This is the dangerous one — it's a direct doorway for exploitation (buffer overflows, injection, logic bypass).

PEN TESTER TAKEAWAY

Whenever you test an input field, always try: special characters (' " ; -- < >), extremely long strings, unexpected data types (letters into a number field), empty input, and unicode/encoded input. If the app crashes, throws a raw stack trace, or behaves strangely — that is a signal of missing/poor error handling, and worth digging into further.

2.2 Lack of Code Signing

Code signing is a way for a developer to *digitally prove* "I am a known, verified author, and I vouch that this code has not been tampered with." Large platform owners (Microsoft, Apple, etc.) run programs where a developer must prove their identity before their software is trusted and signed.

WHY THIS IS A VULNERABILITY, NOT JUST AN INCONVENIENCE

Most regular users never reverse-engineer the software they install — they simply trust it. If an application is **unsigned**, the operating system has no way to confirm who really built it or whether it was modified after release. That means:

- An attacker can copy legitimate-looking software, quietly inject malicious code into it, and redistribute it.
- Because there is no signature to verify, the victim's system can only ask "do you trust this anyway?" — and many users click "Yes" without thinking.

REASONING

Signed code = a cryptographic promise of authorship and integrity. No signature (or a broken/fake one) = no promise. From an attacker's perspective, unsigned software is an easy vehicle for smuggling in malicious functionality disguised as something legitimate (trojanized installers, fake updates, cloned apps).

2.3 Insecure Data Transmission

This is the simplest of the four to describe: data that should be protected is sent or stored in **plain text** instead of being encrypted.

QUESTIONS A DEVELOPER (AND A PEN TESTER) SHOULD ASK

- Is sensitive data (names, dates of birth, credentials) encrypted **at rest** (in the database) and **in transit** (over the network)?

- If encryption is used, who has access to the encryption keys? Are they properly protected?
- Are credentials **hard-coded** into the application source code? (This is a common and serious mistake.)
- Is the same password/credential being **reused** across multiple systems?

WHY THIS IS DANGEROUS

If an attacker can intercept traffic (e.g. on an unencrypted Wi-Fi network) or read a database directly, plain-text sensitive data is handed to them with zero extra effort. Hard-coded credentials found in source code (or in a decompiled binary) can often be reused elsewhere — because, realistically, **password reuse is extremely common**, both by end users and even by developers across systems.

PEN TESTER TAKEAWAY

Always check for: cleartext HTTP instead of HTTPS, unencrypted database fields for sensitive data, hard-coded secrets in config files or app binaries (grep for "password", "apikey", "secret"), and credential reuse between services once you obtain valid credentials from one system.

2.4 Trusting User Input

This is described in the lesson as "the one that gets you every time" — and it ties directly back into error handling. It is fundamentally a **secure coding / Secure Software Development Life Cycle (SDLC)** issue: never assume the user (or attacker) will send you data in the format, type, or length you expect.

EXAMPLE FROM THE LESSON

A "Name" field should reasonably only need somewhere around 50–75 characters. If the developer does not enforce a maximum length, an attacker can feed the field far more data than intended. This can cause the application to behave unexpectedly — a classic entry point for a **buffer overflow**, where excess data overwrites memory the program did not intend to expose to user control.

REASONING

"Trusting user input" and "error handling" are two sides of the same coin: error handling is about *reacting correctly* when bad input arrives; trusting user input is about the deeper mistake of *never validating in the first place*. Every value that

comes from outside the application (a form field, an API parameter, a file upload, a cookie) must be treated as **untrusted** until it is checked.

THE BALANCING ACT — ERROR MESSAGES CAN LEAK TOO MUCH

There is a subtle trade-off here. Error messages need to be useful enough to help a legitimate user understand what went wrong, but not so detailed that they hand an attacker a roadmap for reverse-engineering the application (e.g. full stack traces, internal file paths, database engine version and query syntax). Deciding exactly how much detail to show is described in the lesson as "an art as much as a science" — there is no universal one-size-fits-all rule; it takes testing and judgement.

PEN TESTER TAKEAWAY

This is why **input validation** (whitelisting allowed characters/length/type) and **output/error sanitisation** (never leaking internal details) are both graded heavily in real assessments. When you see a raw, detailed error message from a target application during testing, treat it as free reconnaissance.

SECTION 2 SUMMARY — HOW THESE FOUR CAUSES CONNECT

Good software is not one single fix; it's the combination of all four: validate what comes in (trusting user input), react safely when something is wrong (error handling), prove who built the code and that it wasn't tampered with (code signing), and protect data while it moves and while it's stored (secure transmission). A weakness in any one of these categories is, in practice, exactly what a pen tester goes looking for.

03 Where To Find Public Exploit Code

Once you've identified that a vulnerability exists on a target (through scanning, version enumeration, manual testing, etc.), the next question is: is there already a working exploit for this out in the wild? These are the resources covered in the lesson, roughly in the order they were introduced.

3.1 Exploit-DB

Exploit-DB is described as probably "the big dog on the block" — a website (and project) whose entire purpose, as the name suggests, is hosting a database of publicly submitted exploits.

HOW IT'S USED

- Browse the most recently submitted exploits, each listed with the affected software/version and the date it was added.
- Example entries mentioned in the lesson: a stored Cross-Site Scripting (XSS) and privilege escalation issue in **Dolibarr ERP-CRM 14.0.2**, and an **Unquoted Service Path** vulnerability in "Macro Expert 4.7" that can compromise a Windows system.
- Clicking into an entry usually shows a description of the vulnerability, technical details, and often the actual exploit code or a proof-of-concept.

UNDERSTANDING "UNQUOTED SERVICE PATH" (EXAMPLE WALKTHROUGH)

The lesson quotes the advisory almost directly: a successful attack requires a **local** user to insert their own code into the system root path, undetected by the OS or security tools, positioned so it may execute during application startup or reboot. If successful, the attacker's code runs with the **elevated privileges** of that service. This is a classic Windows local privilege escalation pattern — worth remembering as a technique name, not just a one-off bug.

PRACTICAL NOTES

- Exploit code found online is not always plug-and-play — it is sometimes intentionally obfuscated, broken, or written for a very specific environment/version. Expect to read and adapt scripts, which usually requires basic scripting/programming knowledge.

- Always confirm exploit code is being used only against systems you are **authorized** to test.

3.2 SearchSploit (Offline Exploit-DB)

SearchSploit is the command-line, offline copy of the Exploit-DB database. It ships pre-installed on Kali Linux, which is extremely useful when you don't have (or don't want) live internet access from an engagement network.

COMMAND USED IN THE LESSON

```
searchsploit dolibarr
```

Running this against the software name "Dolibarr" returned multiple results, including SQL Injection, Cross-Site Scripting, Cross-Site Request Forgery, and Remote Code Execution entries — illustrating that a single piece of software can have many different historical vulnerabilities.

REASONING — WHY USE SEARCHSPOIT INSTEAD OF THE WEBSITE

- **Works offline** — useful in isolated lab networks or engagements with restricted internet access.
- **Fast local search** — you can immediately search by software name and version without leaving the terminal.
- It mirrors the same underlying Exploit-DB data, so results line up with what you'd find on the website.

3.3 Vulners

Vulners is a vulnerability and exploit database site. The lesson highlights it as a strong complement to Exploit-DB — sometimes Exploit-DB has an entry that Vulners doesn't, and vice versa, so it's smart to check both ("a one-two punch").

HOW IT'S USED

Search or browse the Vulners database (e.g. for a known exploit family like **EternalBlue**) to reach a page showing the associated **CVE ID**, the **CVSS v3** severity score, and a description. From there, look specifically for the **description** section,

which usually states whether working exploit code is publicly available and how it can be obtained or run.

PEN TESTER TAKEAWAY

Don't rely on a single exploit source. Cross-checking Exploit-DB and Vulners for the same vulnerability increases the chance of finding a working PoC, and gives you the official CVSS scoring to justify severity ratings in your report.

3.4 PacketStorm Security

PacketStorm is described as having "been around a hot minute" — a long-standing, well-established security resource with its own dedicated exploits section.

EXAMPLE FROM THE LESSON

Recent listings included a Cross-Site Scripting issue in a **WordPress "Enfold" theme** and a **SQL Injection** vulnerability in an "Online Motorcycle Bike Rental System 1.0". Opening the entry showed the actual exploit source code (written in Python in this example), with a download option available to save the script directly.

PEN TESTER TAKEAWAY

PacketStorm is good for the same reason Vulners is: another independent source of exploit code and methodology write-ups. Different sites index different submissions, so checking multiple sources is standard practice, not overkill.

3.5 Google Search / Google Dorking

It might not sound like a "hacking tool," but a plain Google search is one of the most effective ways to locate proof-of-concept exploits, technical write-ups, and blog posts about a specific vulnerability.

HOW IT'S USED

Searching a term like `"Dolibarr exploit privilege escalation"` surfaced results from Exploit-DB itself as well as other independent vulnerability-tracking sites (e.g. vulmon.com). Google often finds more obscure corners of the internet that host proof-of-concept (PoC) code not indexed anywhere else — and frequently, Google search results are exactly what lead you to the next resource: GitHub.

REASONING

Search engines index far more of the internet than any single exploit database can maintain. A researcher's personal blog, a university write-up, a forum thread, or a GitHub repo may contain a working exploit long before (or instead of) it ever gets formally submitted to Exploit-DB. Treat search engines as a "meta-resource" that leads you to all the others.

3.6 GitHub

Many researchers and framework maintainers publish exploit code directly as GitHub repositories, including individual proof-of-concept scripts and full framework modules.

EXAMPLE FROM THE LESSON

Searching GitHub for `"eternalblue"` surfaces repositories containing a Metasploit module implementing the EternalBlue exploit inside the Metasploit Framework.

REASONING — WHY THIS MATTERS BEYOND "JUST DOWNLOADING AND RUNNING"

A GitHub repo is not always the only usable version of an exploit — you can also study the source code itself as a learning resource. The lesson makes an important point: you can read someone else's exploit implementation, understand the technique it demonstrates, and then build your own customised version rather than depending on a specific framework like Metasploit. This is a genuinely valuable skill-building habit for a junior pen tester: read real exploit code, don't just execute it blindly.

CAUTION

Code copied from GitHub, forums, or any public source should always be reviewed before execution. Public exploit repositories can contain outdated, broken, or even intentionally malicious code. Read it, understand what it does, and test only in an authorized lab environment (e.g. your own Kali + Metasploitable 2 setup) before ever using it on a real engagement target.

Summary Table — Public Exploit Resources

Resource	Type	Best Used For
Exploit-DB	Website (online DB)	Primary, most comprehensive public exploit archive

SearchSploit	CLI tool (offline)	Fast local/offline search, pre-installed on Kali
Vulners	Website (vuln DB + CVSS)	Cross-checking Exploit-DB, getting CVE/CVSS details
PacketStorm	Website (exploit archive)	Alternative source, often has unique submissions
Google Search	Search engine	Finding write-ups, blogs, and obscure PoCs; a gateway to other sources
GitHub	Code repository host	Full exploit source code, frameworks (e.g. Metasploit modules), learning by reading real code

04 Exploit Chaining

Exploit chaining is not a website or tool — it is a **concept**, and arguably the most important mindset shift for a junior pen tester to internalize.

THE CORE IDEA

Real systems are rarely broken open by a single exploit. In practice, a pen tester often needs to link **multiple** vulnerabilities together, one leading into the next, before finally gaining full access.

EXAMPLE CHAIN FROM THE LESSON

- 1 An **authorization bypass** in a web application gives initial access into the app.
- 2 From inside, a **file upload vulnerability** is discovered, allowing a custom PHP file to be uploaded.
- 3 A PHP filtering mechanism blocks the upload, so the payload is **Base64-encoded** to bypass the filter.
- 4 Once inside a WordPress-based target, a **vulnerable plugin** is found that requires certain existing privileges to exploit — so the attacker works to obtain those privileges first, then uses the plugin flaw to escalate further.

PEN TESTER TAKEAWAY

Sometimes a single exploit does get you straight in — that does happen. But expect the more common case: you'll hit roadblocks, and progress will require combining several smaller weaknesses into one continuous attack path. Document each step carefully in your report; a chained finding (multiple low/medium issues combined into a critical outcome) is often more valuable and convincing to a client than any single isolated bug.

05 Quick Reference & Cheat Sheet

Root Causes of Vulnerabilities — At a Glance

Cause	What It Means	What To Test For
Lack of Error Handling	App doesn't gracefully manage unexpected input/behaviour	Special characters, malformed input, verbose crash messages
Lack of Code Signing	No cryptographic proof of who wrote the code / that it's unmodified	Unsigned installers, tampered/ cloned software
Insecure Data Transmission	Sensitive data sent or stored in plain text	HTTP vs HTTPS, plaintext DB fields, hard-coded credentials
Trusting User Input	No validation of length, type, or format of user-supplied data	Long strings, wrong data types, buffer overflow attempts

Command Reference

```
# Search the local (offline) Exploit-DB mirror on Kali Linux
searchsploit <software-name>

# Example from the lesson
searchsploit dolibarr
```

Checklist — Before You Trust Any Public Exploit

- Confirm you are authorized to test the target system.
- Read the exploit code before running it — understand what it actually does.
- Cross-check the vulnerability across more than one source (Exploit-DB + Vulners + PacketStorm).
- Note the exact affected version, and confirm your target matches it.
- Test in an isolated lab (e.g. Kali + Metasploitable 2 / DVWA) before use on a real assessment.
- Be ready to modify exploit code — public PoCs are often written for a slightly different environment.

06 Key Takeaways for a Junior Pen Tester

1. ROOT CAUSES GIVE YOU PREDICTION POWER

Once you understand that most vulnerabilities trace back to poor error handling, missing code signing, insecure transmission, or blind trust in user input, you can predict where to test even on software you've never seen before.

2. MULTIPLE SOURCES BEAT ONE SOURCE

Exploit-DB, SearchSploit, Vulners, PacketStorm, Google, and GitHub each surface different results. Professional pen testers check more than one before concluding "no public exploit exists."

3. READING CODE IS A SKILL, NOT A SHORTCUT TO SKIP

The lesson repeatedly emphasizes reading and understanding exploit code — not just running it. This is what separates a script kiddie from a competent pen tester: the ability to read a PoC, understand the underlying technique, and adapt it.

4. REAL ENGAGEMENTS ARE RARELY "ONE EXPLOIT AND DONE"

Expect to chain vulnerabilities together. Build the habit of documenting every step of a chain clearly — this is exactly what clients pay for in a penetration test report.

"You're constantly hitting roadblocks... you might not just be a one-size-fits-all. You might need a couple of things to get to the mountain top." — Daniel Lowrie, on exploit chaining

NEXT STEPS FOR YOUR OWN PRACTICE

- Pick one CVE from your CyberStr lab targets (Metasploitable 2, DVWA, Mutillidae) and try to find it independently across Exploit-DB, SearchSploit, and Vulners.
- Practice reading one GitHub PoC line-by-line and writing a short explanation of what each part does — this builds toward writing your own exploits later.
- Add a "Root Cause" column to your own engagement report template, tagging each finding as Error Handling / Code Signing / Data Transmission / Input Trust — this shows report reviewers deeper analytical thinking, not just tool output.